

EduPro Analytics Dashboard: A Data-Driven Framework for Analyzing Learner Demographics and Course Enrollment Behavior Using Python and Streamlit

ARITRA DAS

B.Tech CSE AI&ML, THE NEOTIA UNIVERSITY

Project Report

Emial:aritradas167@gmail.com

Abstract

Educational institutions increasingly rely on data to understand who their learners are and how they engage with course offerings, yet many platforms still present enrollment data in static spreadsheets that are difficult to interpret at a glance. This paper presents EduPro Analytics Dashboard, an interactive web-based analytics tool built with Python, Pandas, Plotly, and Streamlit that transforms raw learner and enrollment records into actionable visual insights. The dashboard was designed around a structured workflow of data cleaning, feature engineering, and interactive visualization, enabling administrators to filter learner data by age group, gender, course category, and course level, and to view key performance indicators such as total users, total courses, total enrollments, and average courses per user. The system was implemented using a lightweight Python stack, version-controlled with Git and GitHub, and deployed publicly through Streamlit Community Cloud, making it accessible without local installation. Exploratory analysis performed through the dashboard revealed patterns in age-group participation, category-wise popularity, and enrollment trends over time, demonstrating how interactive filtering surfaces insights that static reports tend to obscure. The results show that a low-cost, open-source technology stack can deliver a usable and extensible analytics solution suitable for small and medium educational platforms. The paper also identifies current limitations, including reliance on a static dataset and the absence of predictive modeling, and proposes future extensions involving machine learning-based enrollment prediction, database integration, and user authentication. This work contributes a practical reference implementation for learning analytics dashboards and a discussion of the design choices involved in building them with modern Python tooling.

Keywords — Educational Analytics, Streamlit, Python, Dashboard, Data Visualization, Enrollment Analysis, Pandas, Learning Analytics

1. Introduction

The rapid expansion of online and blended learning has produced an unprecedented volume of data describing who learners are, what courses they choose, and how their engagement changes over time. Learning management systems, massive open online course platforms, and institutional enrollment databases now generate records at a scale that is impossible to interpret manually. This has given rise to the field of learning analytics, which is broadly concerned with the measurement, collection, analysis, and reporting of data about learners for the purpose of understanding and optimizing learning and the environments in which it occurs. While the volume of educational data has grown, the tools available to non-technical stakeholders — administrators, program coordinators, and instructors — to make sense of that data have not kept pace. Enrollment figures are frequently locked inside spreadsheets or buried in administrative software that was never designed for exploratory analysis, forcing decision-makers to rely on static, backward-looking reports rather than dynamic, exploratory views of their own data.

Data-driven decision making has become a guiding principle across nearly every sector, and education is no exception. Institutions that can identify which age groups are underrepresented in certain course categories, which course levels attract the most sustained interest, or how enrollment volume shifts across a semester are better positioned to allocate resources, design targeted outreach, and adjust curricula. However, the value of data-driven decision making depends entirely on the accessibility of the underlying data. A dataset that requires a data analyst to query and a data scientist to visualize is of limited use to a program coordinator who needs a same-day answer to a simple question such as “which course category is growing fastest among learners under twenty-five?” Interactive dashboards close this gap by allowing non-technical users to explore data themselves, applying filters and drilling into subsets without writing a single line of code.

Dashboard applications have matured considerably over the past decade, moving from static business intelligence tools that require licensing and dedicated infrastructure to lightweight, open-source frameworks that can be built and deployed by a single developer in a matter of days. Among these, Streamlit has emerged as a particularly accessible option for the Python data science community, allowing developers to convert a data analysis script into an interactive web application with minimal boilerplate. Combined with Pandas for data manipulation and Plotly for interactive charting, Streamlit enables the rapid construction of dashboards that would previously have required a dedicated front-end framework and a separate backend service.

Despite the availability of these tools, there remains a shortage of documented, end-to-end reference implementations that walk through the design and construction of a learner demographics dashboard specifically — as opposed to general-purpose business dashboards. Most public tutorials focus narrowly on a single chart type or a single Streamlit feature, rather than the complete pipeline from raw enrollment data to a deployed, filterable analytics application. This gap creates a barrier for students and early-career developers who understand the individual

building blocks — Pandas, Plotly, Streamlit — but have not seen them integrated into a coherent, deployable system aimed at a genuine educational use case.

The need for learner demographic analysis specifically is motivated by a simple observation: course content and course delivery are rarely one-size-fits-all. Learners of different age groups, and learners enrolling in different course categories or levels, tend to exhibit different engagement patterns. An institution that treats its entire learner base as a single undifferentiated group risks designing courses, marketing campaigns, and support services that fail to serve important subpopulations well. Demographic-aware analytics, presented through an interactive interface, allows an institution to see its learner base as it actually is — segmented, uneven, and dynamic — rather than as an aggregate statistic.

Current challenges in this space include the fragmentation of learner data across multiple systems, the technical barrier separating raw data from usable insight, the cost of commercial business intelligence tooling, and the general absence of lightweight, purpose-built solutions for smaller institutions that cannot justify enterprise software licenses. Many existing learning analytics dashboards described in the literature are either proprietary, tied to a specific learning management system, or designed for large-scale institutional deployment with correspondingly large implementation costs. There is comparatively little written about small-footprint, open-source dashboards that a student or a small team can build, deploy for free, and iterate on quickly.

This paper addresses that gap by presenting EduPro Analytics Dashboard, a project developed to analyze learner demographics and course enrollment behavior using an open-source Python stack. The remainder of this paper is organized as follows. Section 2 reviews related work in learning analytics, dashboard systems, and educational data mining. Section 3 states the problem this project addresses. Section 4 lists the project’s objectives. Section 5 describes the methodology, including the data pipeline from collection to deployment. Section 6 describes the system architecture. Section 7 details the implementation. Section 8 discusses the results obtained from the dashboard. Sections 9 and 10 discuss the advantages and limitations of the system, respectively. Section 11 outlines future scope, and Section 12 concludes the paper.

2. Literature Review

Learning analytics as a distinct field emerged from the convergence of educational data mining, academic analytics, and the broader movement toward evidence-based decision making in education. Learning analytics is generally understood as the measurement, collection, analysis, and reporting of data about learners and their contexts for the purpose of understanding and optimizing learning and the environments in which it occurs, and the field has grown rapidly as institutions have sought to apply data-driven methods to improve educational outcomes. A recurring theme in this literature is the distinction between analytics performed for institutional or administrative purposes and analytics performed to directly support learners and instructors; dashboards of the kind developed in this project sit closer to the administrative and program-

management end of that spectrum, since their primary users are coordinators and decision-makers rather than students themselves.

A closely related field, educational data mining, focuses on developing methods for exploring data originating from educational settings and using those methods to better understand students and the environments in which they learn. Surveys of this field have catalogued a wide range of applications, including student modeling, prediction of performance, detection of undesirable behaviors, grouping students, social network analysis, and generating recommendations, and have noted that visualization for instructors and administrators is itself one of the recognized application areas of educational data mining, distinct from purely predictive tasks. This distinction is important for the present project: EduPro Analytics Dashboard is deliberately positioned as a descriptive and exploratory tool rather than a predictive one, focusing on making existing demographic and enrollment patterns visible rather than forecasting future behavior. Later work in the same area has emphasized that data mining in education is not a single method but a family of techniques applied across very different data types, from clickstream logs to standardized test scores, reinforcing the idea that dashboard design must be tailored to the specific data available rather than treated as a generic problem.

Dashboard systems, as a category of software independent of the educational domain, have been studied extensively within the information visualization and business intelligence communities. A widely cited definition characterizes a dashboard as a visual display of the most important information needed to achieve one or more objectives, consolidated and arranged on a single screen so it can be monitored at a glance. This definition is useful for evaluating EduPro Analytics Dashboard's design, since the project explicitly organizes its key performance indicators — total users, total courses, total enrollments, and average courses per user — into a single top-level view before allowing the user to drill down through filters, mirroring the “glanceable summary followed by detail” pattern recommended in the dashboard design literature. The same body of work cautions against common design failures such as overcrowding a dashboard with unrelated metrics, using chart types that misrepresent the underlying data, and failing to provide context for the numbers displayed; these cautions directly informed the decision to limit the dashboard's KPI cards to four core metrics and to pair each visualization with a specific, filterable question rather than displaying every possible cross-tabulation at once.

Within the education-specific dashboard literature, learning analytics dashboards have been described as tools that visualize learning traces to support self-reflection or awareness for learners, teachers, or administrators. Reviews of learning dashboard applications have observed that most published dashboards target either learners themselves, for self-monitoring, or instructors, for monitoring a single class, and that far fewer target institutional or program-level administrators looking across an entire learner population — the gap that this project attempts to address. Related work on attention and awareness in learning analytics has argued that visualization is not merely a reporting mechanism but an intervention in its own right, since simply making patterns visible can change the behavior of the people who see them; this argument supports the broader premise

of the present project, namely that an interactive dashboard has value even without any predictive or prescriptive component, purely by making demographic and enrollment structure visible to decision-makers who previously lacked easy access to it.

Student performance analysis is one of the most mature strands of educational data mining, encompassing everything from early-warning systems that flag at-risk students to fine-grained analyses of how specific behaviors correlate with grades. While EduPro Analytics Dashboard does not model performance directly, this strand of literature is relevant because it establishes the broader analytical vocabulary — cohorts, engagement metrics, dropout risk, category-level comparisons — that demographic dashboards like this one are typically built to feed into. A natural extension of the current project, discussed further in Section 11, would connect its descriptive demographic view to a downstream performance-prediction model, following the pattern established in this literature.

Finally, the data visualization literature more broadly has shaped the specific chart choices made in this project. Guidance on visual encoding generally favors simple, well-understood chart types — bar charts for categorical comparison, line charts for trends over time, and heatmaps for two-dimensional relationships — over more elaborate visual forms, on the grounds that simple charts are read more accurately and more quickly by a general audience. This guidance is reflected directly in the dashboard’s choice of bar charts for course category and course level distribution, a line chart for enrollment trends over time, and a heatmap for cross-tabulating categorical variables such as age group against course category.

Taken together, the reviewed literature supports three design decisions made in this project: first, that a descriptive, non-predictive dashboard is a legitimate and useful category of learning analytics tool in its own right; second, that dashboards aimed at administrators and program coordinators, rather than individual learners or instructors, remain comparatively underexplored and therefore worth documenting; and third, that simple, well-established chart types are preferable to more elaborate visualizations when the audience includes non-technical decision-makers.

3. Problem Statement

Educational institutions collect substantial amounts of data describing who their learners are and how they enroll in courses, but this data is rarely presented in a form that supports quick, exploratory decision-making. Current educational platforms often lack interactive analytical dashboards that provide insights into learner demographics and enrollment behavior. Enrollment records are typically stored as raw tables — in spreadsheets, learning management system exports, or administrative databases — that require manual filtering, pivoting, and charting before they yield any usable insight. This manual process is time-consuming, error-prone, and inaccessible to staff without spreadsheet or programming expertise. As a result, decisions about course offerings, marketing focus, and resource allocation are frequently made on the basis of

intuition or outdated summary reports rather than current, disaggregated data. There is a clear need for a lightweight, interactive tool that allows non-technical staff to explore learner demographic and enrollment data directly, filtering by attributes such as age group, gender, course category, and course level, without requiring any programming knowledge on the part of the end user.

4. Objectives

The objectives of this project are as follows:

1. To design and implement an interactive dashboard that visualizes learner demographic data and course enrollment behavior.
2. To provide filterable views of the dataset by age group, gender, course category, and course level, enabling targeted exploration of subpopulations.
3. To summarize the dataset at a glance through key performance indicator cards, including total users, total courses, total enrollments, and average courses per user.
4. To implement a reliable data preprocessing pipeline that handles missing values, duplicate records, and feature engineering, including the derivation of age groups from raw age values.
5. To generate interactive visualizations — including distribution charts, trend charts, and a heatmap — that allow patterns to be identified without manual data manipulation.
6. To provide a mechanism for exporting filtered results as a downloadable CSV report.
7. To deploy the completed dashboard to a publicly accessible cloud platform, removing the need for local installation by end users.
8. To document a complete, reproducible workflow from raw dataset to deployed application using an open-source Python technology stack.

5. Methodology

The project followed a linear, iterative workflow consisting of seven stages: dataset collection, data cleaning, exploratory data analysis, feature engineering, dashboard development, visualization, and deployment. Each stage is described below.

Dataset Collection. The starting point of the project was an educational dataset containing, for each record, a unique user identifier, learner age, gender, a course identifier, course category, course level, and enrollment date. This structure was chosen because it captures both the demographic attributes needed to segment learners and the enrollment attributes needed to analyze course-level behavior, while remaining simple enough to be representative of the kind of export a real institution could plausibly produce from its own systems.

Data Cleaning. Raw educational datasets are rarely analysis-ready. The cleaning stage addressed three categories of issues. First, missing value handling: rows with missing critical fields, such as user identifier or enrollment date, were flagged, and where appropriate, missing categorical values were imputed with an explicit “Unknown” label rather than silently dropped, so that the extent of missingness itself remained visible to dashboard users rather than being hidden. Second, duplicate removal: enrollment records that were exact duplicates across all fields were identified and removed to avoid inflating enrollment counts. Third, type and format standardization: date fields were parsed into a consistent datetime format, categorical text fields such as course category and gender were normalized in case and spacing to prevent the same category from being counted as two distinct categories due to inconsistent capitalization.

Exploratory Data Analysis. Before building any visual component, the cleaned dataset was explored using Pandas to understand its basic statistical properties: the distribution of ages, the balance of gender categories, the relative frequency of course categories and levels, and the time span covered by the enrollment dates. This exploratory step directly informed which filters and visualizations would be most useful in the final dashboard, ensuring that the dashboard’s design was grounded in the actual shape of the data rather than a generic assumption about what an educational dataset should contain.

Feature Engineering. Several derived features were constructed to support the dashboard’s filters and visualizations. The most significant of these was age group creation, in which the continuous age variable was binned into a small number of interpretable categories, allowing users to filter and compare cohorts such as “under 18,” “18–24,” “25–34,” and so on, rather than filtering on exact age values. Additional derived fields included enrollment counts per user, used to compute the average-courses-per-user KPI, and enrollment month or period fields, used to support the enrollment trends visualization.

Dashboard Development. The dashboard itself was developed using Streamlit, a Python framework that converts a linear data-processing script into a reactive web application. The development process began with a minimal version of the application displaying the raw dataset as a table, followed by the incremental addition of KPI cards, filter widgets, and visualizations, with each addition tested locally before being layered onto the previous version. This incremental approach reduced the risk of a single large, difficult-to-debug implementation and made it straightforward to isolate the source of any rendering or performance issue as it arose.

Visualization. Once the dashboard’s data pipeline was stable, Plotly was used to construct the project’s interactive charts, including bar charts for course category and course level distribution, a bar or line chart for age group analysis, a line chart for enrollment trends over time, and a heatmap summarizing the relationship between two categorical variables at once, such as age group against course category. Plotly was selected over static plotting libraries specifically because its charts respond to hover interactions and can be embedded directly into a Streamlit application with a single function call, preserving interactivity without additional front-end code.

Deployment. The completed application was version-controlled using Git, with the repository hosted on GitHub to track changes and to serve as the source repository for deployment. The application was then deployed to Streamlit Community Cloud, which builds and serves a Streamlit application directly from a connected GitHub repository. This deployment approach was chosen because it requires no separate server provisioning, no manual dependency installation on a production machine, and no cost for a project of this scale, making it well suited to an internship-level project intended for demonstration and evaluation rather than large-scale production use.

6. System Architecture

The system architecture of EduPro Analytics Dashboard follows a straightforward client-facing, single-tier design appropriate for a small-scale analytics tool. At the center of the architecture is the dataset, stored as a structured file containing the fields described in Section 5. This dataset is loaded and processed by Pandas, which performs the cleaning and feature engineering steps described above and produces a clean, in-memory data structure that the rest of the application relies on.

Above the data-processing layer sits the Streamlit application layer, which is responsible for three things: rendering the user interface, capturing user input from the filter widgets, and orchestrating the flow of data from the Pandas layer to the visualization layer in response to that input. Because Streamlit follows a reactive execution model, any change to a filter — for example, selecting a different course category — triggers the script to re-run with the new filter state, and Pandas re-filters the underlying dataset accordingly. The filtered subset is then passed to the visualization layer, implemented with Plotly, which renders the corresponding charts.

The user interacts with the system entirely through a web browser, which communicates with the Streamlit server hosted on Streamlit Community Cloud. From the user’s perspective, the architecture is a single web page containing filter controls at the top or side, KPI cards summarizing the currently filtered subset, and a series of charts and a data table below. No database server or separate backend API is required, since Pandas operates directly on the dataset file loaded into the application’s memory at runtime.

(Figure 1 placeholder: architecture diagram showing User → Web Browser → Streamlit Application (Filters, KPI Cards) → Pandas Data Processing Layer → Dataset → Plotly Visualization Layer → rendered back to the browser.)

This architecture, while simple, is well matched to the project’s scope. It avoids the operational overhead of a multi-tier system with a dedicated database and API layer, while still cleanly separating data processing (Pandas), presentation logic (Streamlit), and visualization (Plotly) into distinct, replaceable components — a separation that will be important if the system is later extended, as discussed in Section 11, to use a proper database backend or a machine learning prediction service.

7. Implementation

The implementation of EduPro Analytics Dashboard was carried out entirely in Python, chosen for its dominant position in the data analysis ecosystem and for the availability of mature, well-documented libraries covering every stage of the pipeline described in Section 5.

Python Implementation. The application was structured as a single main Streamlit script that imports the dataset, applies the cleaning and feature engineering functions, defines the filter widgets, computes the KPI values, and renders the visualizations, in that order. Helper functions were factored out for the more complex operations — in particular, age-group binning and KPI computation — to keep the main script readable and to make those functions independently testable.

Pandas Preprocessing. Pandas was used for every data-manipulation step in the pipeline: reading the source dataset into a DataFrame, dropping or imputing missing values, removing duplicate rows, parsing enrollment dates into datetime objects, and creating the derived age-group column using Pandas’ binning functionality. Pandas’ groupby and aggregation functions were used extensively to compute the summary statistics that feed both the KPI cards and the distribution charts, taking advantage of Pandas’ vectorized operations for efficiency on datasets of this size.

KPI Calculation. Four KPI cards were implemented: total users, computed as the count of unique user identifiers in the currently filtered dataset; total courses, computed as the count of unique course identifiers; total enrollments, computed as the count of enrollment records; and average courses per user, computed by dividing total enrollments by total users. Because these calculations were applied to the filtered subset of the data rather than the full dataset, the KPI cards update dynamically whenever the user changes a filter, giving an immediate numerical summary of whichever subpopulation is currently selected.

Dashboard Filters. Four filter widgets were implemented using Streamlit’s selection components: a multi-select or dropdown for age group, a selector for gender, a selector for course category, and a selector for course level. Each filter widget was bound to the underlying DataFrame such that selecting a value immediately narrows the dataset used for every downstream KPI and chart, implementing the “select and drill down” interaction pattern common to dashboard interfaces.

Plotly Visualizations. Each visualization was implemented as a distinct Plotly Express or Plotly Graph Objects call embedded into the Streamlit layout using Streamlit’s chart-rendering function. The course category and course level distribution charts were implemented as bar charts, ranking categories by enrollment count. The age group analysis chart was implemented as a bar chart segmented by age group. The enrollment trends chart was implemented as a line chart plotting enrollment counts against time, aggregated by month. The heatmap visualization was implemented by pivoting the dataset into a two-dimensional count matrix, for example age group against course category, and rendering it with Plotly’s heatmap function to reveal cross-cutting patterns that a single-variable chart cannot show.

Streamlit Deployment. The application’s dependencies were captured in a requirements file listing the exact Python packages used, including Streamlit, Pandas, and Plotly, ensuring that the deployment environment on Streamlit Community Cloud could reproduce the local development environment exactly. The Streamlit Community Cloud platform was then connected to the project’s GitHub repository, and the application was configured to redeploy automatically whenever changes were pushed to the main branch.

GitHub Integration. Git was used throughout development to track incremental changes, with commits corresponding to discrete milestones such as “add age group filter” or “add enrollment trend chart.” Hosting the repository on GitHub served two purposes: it provided a version history that could be reviewed to understand how the project evolved, and it served as the direct source for Streamlit Community Cloud’s continuous deployment process, meaning that no separate deployment script or manual upload step was required.

(Table 1 placeholder: summary of technologies used, mapped to their role in the implementation — Python, Pandas, Streamlit, Plotly, Git, GitHub, Streamlit Community Cloud.)

(Figure 2 placeholder: dashboard screenshot showing filter panel, KPI cards, and primary charts.)

(Figure 3 placeholder: dashboard screenshot showing the heatmap visualization.)

8. Results and Discussion

Exploring the dataset through the completed dashboard produced several observable patterns, discussed here in narrative form rather than as fabricated precise statistics, since the underlying dataset used for development was a representative educational dataset rather than data drawn from a live institution.

Age group participation. Filtering by age group showed that enrollment activity was not evenly distributed across the learner population; certain age bands, generally corresponding to early-career and university-age learners, accounted for a disproportionately large share of total enrollments relative to their share of total users, suggesting that these groups tend to enroll in multiple courses rather than a single course. This kind of pattern would be difficult to notice in a raw spreadsheet but became immediately visible once the age-group bar chart was filtered and compared against the “average courses per user” KPI.

Gender participation. Comparing enrollment volume across gender categories, after filtering, showed differences in both the total number of learners and the distribution of course categories chosen, with some course categories showing a more balanced split than others. The dashboard’s ability to apply a gender filter and immediately observe how the course-category distribution chart reshapes itself was one of the clearer demonstrations of the value of interactive, as opposed to static, reporting.

Popular course categories. The course category distribution chart consistently highlighted a small number of categories accounting for a large share of total enrollments, with a longer tail of less popular categories. This “few categories dominate” pattern is common in enrollment data generally and is useful for administrators deciding where to focus marketing or curriculum development effort.

Course level preferences. The course level distribution chart showed that introductory-level courses tended to attract more total enrollments than advanced-level courses, which is an expected pattern given that a smaller subset of learners typically progresses to advanced material, but the dashboard made it possible to check whether this pattern held consistently across every course category or only in a subset of them, by combining the course level filter with the category-level chart.

Enrollment trends over time. The enrollment trends line chart revealed periods of higher and lower enrollment activity across the time span covered by the dataset. Because the chart could be viewed for the full dataset or for a single filtered subgroup, it was possible to distinguish between overall seasonal patterns affecting all learners and patterns specific to a single age group or course category.

Dashboard usability. From a usability standpoint, the combination of a small number of prominent KPI cards at the top of the page with detailed charts below matched the “glanceable summary, then detail” pattern discussed in Section 2, and informal testing with the completed dashboard suggested that a first-time user could locate a specific insight, such as the most popular course category among a particular age group, in a small number of clicks without any prior explanation of the tool.

Benefits of interactive filtering. The clearest overall finding from working with the dashboard was that interactive filtering surfaces relationships between variables that are not visible in any single static chart or table. A static report might show course category distribution for the entire learner base, but only an interactive dashboard makes it equally easy to ask the same question for learners under twenty-five, or for a specific gender, or for a specific course level, without requiring a new report to be generated each time. This flexibility is the central practical contribution of the dashboard relative to the static reporting it is intended to replace.

9. Advantages

The EduPro Analytics Dashboard offers several advantages over static, spreadsheet-based reporting:

- **Easy analysis:** Complex demographic and enrollment questions can be answered through a few clicks rather than manual spreadsheet manipulation.
- **Interactive dashboard:** Filters and charts update dynamically, allowing exploratory, iterative questioning of the data rather than a single fixed view.

- **Better decision making:** KPI cards and visualizations that respond to filters give administrators an evidence-based foundation for decisions about course offerings and outreach.
- **User friendly:** The interface requires no programming or spreadsheet expertise, making it accessible to non-technical staff.
- **Cloud accessibility:** Deployment on Streamlit Community Cloud means the dashboard can be accessed from any device with a web browser, without local installation.
- **Fast filtering:** Pandas' vectorized operations allow filters to be applied and charts to be redrawn with minimal delay, even as filter combinations change.
- **Real-time interaction:** The reactive nature of Streamlit means the dashboard behaves as a live tool for exploration rather than a static, pre-rendered report.

10. Limitations

Despite its advantages, the current implementation of EduPro Analytics Dashboard has several limitations that should be acknowledged:

- **Static dataset:** The dashboard currently operates on a fixed dataset loaded at startup rather than a live, continuously updated data source, meaning new enrollments are not reflected until the underlying dataset file is manually refreshed and redeployed.
- **No predictive analytics:** The system is descriptive rather than predictive; it does not forecast future enrollment trends or identify learners at risk of disengagement.
- **No authentication:** The dashboard does not currently implement user authentication or role-based access control, which would be necessary before deploying it with real, sensitive learner data in a production institutional setting.
- **Limited scalability:** Because the dataset is loaded entirely into memory using Pandas, the current architecture is well suited to small and medium datasets but would require a different data-processing approach for very large-scale institutional deployments.
- **No real-time database integration:** The dashboard reads from a static file rather than querying a live database, which limits its usefulness as an operational, continuously updated monitoring tool.

11. Future Scope

Several extensions could meaningfully increase the value of the EduPro Analytics Dashboard:

- **Machine learning prediction:** Incorporating a predictive model trained on historical enrollment data could allow the dashboard to forecast future enrollment volume by course category or age group.

- **AI recommendations:** A recommendation component could suggest courses to learners, or suggest under-marketed categories to administrators, based on demographic and enrollment patterns.
- **Student performance prediction:** Extending the dataset to include performance or completion data would allow the dashboard to be connected to the broader body of work on predicting student outcomes.
- **SQL database integration:** Replacing the static dataset file with a proper relational database would allow the dashboard to reflect enrollments as they occur, rather than requiring manual data refreshes.
- **Real-time analytics:** Building on database integration, the dashboard could be extended to poll or subscribe to new enrollment events and update its visualizations without a full page reload.
- **Mobile dashboard:** A dedicated mobile-responsive layout, or a companion mobile application, would extend the dashboard’s accessibility beyond desktop and laptop browsers.
- **User authentication:** Adding authentication and role-based access control would be a prerequisite for any deployment involving real, potentially sensitive learner data.
- **Advanced visualization:** Additional visualization types, such as cohort-retention charts or geographic maps of learner distribution, could be added as the underlying dataset grows richer.

12. Conclusion

This paper presented EduPro Analytics Dashboard, an interactive analytics tool for examining learner demographics and course enrollment behavior, built using Python, Pandas, Plotly, and Streamlit and deployed via Streamlit Community Cloud. The project addressed a genuine gap in many educational settings: the absence of an accessible, interactive way to explore learner and enrollment data without specialized technical skills. By following a structured pipeline — dataset collection, cleaning, exploratory analysis, feature engineering, dashboard development, visualization, and deployment — the project produced a working dashboard offering filterable views by age group, gender, course category, and course level, summarized through four key performance indicators and a set of interactive charts including distribution charts, a trend chart, and a heatmap. The results of exploring the dataset through the dashboard demonstrated the practical value of interactive filtering, which surfaced demographic and category-level patterns that would be difficult to detect in a static report, and confirmed that the “glanceable summary followed by detailed drill-down” design pattern recommended in the dashboard-design literature translates well into a lightweight, open-source implementation. At the same time, the project has clear limitations, most notably its reliance on a static dataset, the absence of predictive

capabilities, and the lack of authentication, which together mean the current version is best understood as a demonstrable prototype rather than a production-ready institutional system. The future extensions outlined in this paper — database integration, predictive modeling, authentication, and mobile accessibility — describe a realistic path from this prototype toward a more complete institutional analytics tool. More broadly, this work contributes a documented, reproducible example of how a modern, low-cost Python stack can be used to build a genuinely useful learning analytics dashboard, and it is hoped that the design decisions and workflow described here will be useful to other students and small teams undertaking similar projects.

References

- [1] G. Siemens, “Learning Analytics: The Emergence of a Discipline,” *American Behavioral Scientist*, vol. 57, no. 10, pp. 1380–1400, 2013.
- [2] R. S. J. d. Baker and K. Yacef, “The State of Educational Data Mining in 2009: A Review and Future Visions,” *Journal of Educational Data Mining*, vol. 1, no. 1, pp. 3–17, 2009.
- [3] C. Romero and S. Ventura, “Educational Data Mining: A Review of the State of the Art,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 40, no. 6, pp. 601–618, 2010.
- [4] C. Romero and S. Ventura, “Data Mining in Education,” *WIREs Data Mining and Knowledge Discovery*, vol. 3, no. 1, pp. 12–27, 2013.
- [5] D. Gašević, S. Dawson, and G. Siemens, “Let’s Not Forget: Learning Analytics Are About Learning,” *TechTrends*, vol. 59, no. 1, pp. 64–71, 2015.
- [6] K. Verbert, E. Duval, J. Klerkx, S. Govaerts, and J. L. Santos, “Learning Analytics Dashboard Applications,” *American Behavioral Scientist*, vol. 57, no. 10, pp. 1500–1509, 2013.
- [7] E. Duval, “Attention Please! Learning Analytics for Visualization and Recommendation,” in *Proc. 1st Int. Conf. Learning Analytics and Knowledge (LAK ’11)*, 2011, pp. 9–17.
- [8] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*. Sebastopol, CA: O’Reilly Media, 2006.
- [9] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, 2010, pp. 56–61.
- [10] W. McKinney, *Python for Data Analysis*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2017.
- [11] J. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O’Reilly Media, 2016.
- [12] Python Software Foundation, “The Python Language Reference,” 2024. [Online]. Available: <https://docs.python.org/3/reference/>

- [13] The pandas development team, “pandas documentation,” 2024. [Online]. Available: <https://pandas.pydata.org/docs/>
- [14] Streamlit Inc., “Streamlit Documentation,” 2024. [Online]. Available: <https://docs.streamlit.io>
- [15] Plotly Technologies Inc., “Plotly Python Open Source Graphing Library,” 2024. [Online]. Available: <https://plotly.com/python/>
- [16] NumPy Developers, “NumPy Documentation,” 2024. [Online]. Available: <https://numpy.org/doc/>
- [17] GitHub Inc., “GitHub Docs,” 2024. [Online]. Available: <https://docs.github.com>
- [18] Git, “Git Documentation,” 2024. [Online]. Available: <https://git-scm.com/doc>
- [19] Streamlit Inc., “Streamlit Community Cloud Documentation,” 2024. [Online]. Available: <https://docs.streamlit.io/streamlit-community-cloud>